

SOC/L1 - 8.2. Detecting Web Attack

Task 1 - Introduction

Web attacks are among the most common ways attackers gain entry into target systems. Public-facing websites and web applications often sit in front of databases and other infrastructure, which are appealing targets for attackers.

Task 2 - Client-Side Attacks

Client-side attacks rely on abusing weaknesses in user behavior or on the user's device. These attacks often exploit vulnerabilities in browsers or trick the user into performing unsafe actions to gain access to accounts and steal sensitive information. Valuable data can be stored on a user's device, so a successful client-side attack can result in data loss.

Common Client-Side Attacks

- **Cross-Site Scripting** (XSS) is the most common(opens in new tab) client-side attack, in which malicious scripts are run in a trusted website and executed in the user's browser. If your website has a comment box that doesn't filter input, an attacker could post a comment like: `Hello <script>alert('You have been hacked'); </script>`. When visitors load the page, the script runs inside their browser, and the pop-up appears. In a real attack, instead of a harmless pop-up, the attacker could steal cookies or session data.
- **Cross-Site Request Forgery** (CSRF): The browser is tricked into sending unauthorized requests on behalf of the trusted user.
- **Clickjacking**: Attackers overlay invisible elements on top of legitimate content, making users believe they are interacting with something safe.

Answer the questions below

What class of attacks relies on exploiting the user's behavior or device? Client-Side

What is the most common client-side attack? XSS

Task 3 - Server-Side Attacks

Server-side attacks exploit backend systems and code flaws—such as poorly secured user input forms—to directly breach databases, steal sensitive data, and compromise web application servers.

Catching Server-Side Attacks: Defenders can catch server-side attacks by tracking the digital evidence left behind in server logs and network traffic.

Common Server-Side Attacks

- **Brute-force** attacks occur when an attacker repeatedly attempts different usernames or passwords in an attempt to gain unauthorized access to an account. Automated tools are often used to send these requests quickly, allowing attackers to go through large lists of credentials and common passwords.
- **SQL Injection** (SQLi) relies on attacking the database that sits behind a website and occurs when applications build queries through string concatenation instead of using parameterized queries, allowing attackers to alter the intended SQL command and access or manipulate data.
- **Command Injection** is a [common attack\(opens in new tab\)](#) that occurs when a website takes user input and passes it to the system without checking it. Attackers can sneak in commands, making the server run them with the same permissions as the application.

Answer the questions below

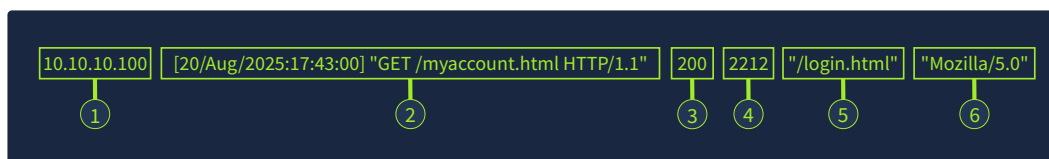
What class of attacks relies on exploiting vulnerabilities within web servers?
Server-Side

Which server-side attack lets attackers abuse forms to dump database contents? SQLi

Task 4 - Log-Based Detection

Access Log Format:

Log Field	Example Indicator
1. Client IP Address	A known malicious or outside of the expected geo range
2. Timestamp and Requested Page	Requests made at unusual hours or repeated in a short period of time
3. Status Code	Repeated 404 responses indicating a page could not be found
4. Response Size	Significantly smaller or larger than normal response sizes
5. Referrer	Referring pages that don't fit normal site navigation
6. User-Agent	Outdated browser versions or common attack tools (e.g. sqlmap , wpscan)



Attacks in Logs

1. The attacker tests for potential directories and forms to exploit with a directory fuzz. The **200** response codes signify valid finds for the attacker.
2. Next, the attacker exploits the **login.php** form found with a brute-force attack. Note the repeated **POST** requests in quick succession. The last **POST** request has a different response code, **302 Found**, which, in this case, signifies a successful login attempt. The page then redirects to the user's account page at **/account**.
3. Once the attacker gains access to the account, they attempt two SQLi payloads, **' OR '1'='1** and **1' OR 'a'='a**, on the **/search** form. If the application builds SQL queries dynamically instead of using parameterised queries, the attacker could dump the database.

```
10.10.10.100 [12/Aug/2025:05:45:26] "GET /testdir HTTP/1.1" 404 273 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:26] "GET /backup HTTP/1.1" 404 273 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:26] "GET /private HTTP/1.1" 404 273 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:26] "GET /catalog HTTP/1.1" 200 8573 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:26] "GET /wordpress HTTP/1.1" 200 10972 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:28] "GET /wp-console HTTP/1.1" 404 273 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:28] "GET /wp-admin HTTP/1.1" 404 273 "gobuster/1.9"
10.10.10.100 [12/Aug/2025:05:45:28] "GET /login.php HTTP/1.1" 200 1807 "gobuster/1.9"

10.10.10.100 [12/Aug/2025:05:47:30] "POST /login.php HTTP/1.1" 401 4937 "Hydra"
10.10.10.100 [12/Aug/2025:05:47:30] "POST /login.php HTTP/1.1" 401 4937 "Hydra"
10.10.10.100 [12/Aug/2025:05:47:30] "POST /login.php HTTP/1.1" 401 4937 "Hydra"
10.10.10.100 [12/Aug/2025:05:47:30] "POST /login.php HTTP/1.1" 401 4937 "Hydra"
10.10.10.100 [12/Aug/2025:05:47:30] "POST /login.php HTTP/1.1" 302 5036 "Hydra"
10.10.10.100 [12/Aug/2025:05:47:30] "GET /account HTTP/1.1" 200 12345 "Hydra"

10.10.10.100 [12/Aug/2025:05:48:13] "GET /search?q=' OR '1'='1&Submit=Submit HTTP/1.1" 200 1861 "sqlmap/1.7.2"
10.10.10.100 [12/Aug/2025:05:49:24] "GET /search?q=1' OR 'a'='a&Submit=Submit HTTP/1.1" 200 1861 "sqlmap/1.7.2"
```

Log Limitations

```
10.10.10.100 [12/Aug/2025:14:32:10] "POST /login HTTP/1.1" 200 532 "/home.html"
"Mozilla/5.0"
```

The above sample log shows the request method, the requested page, and the status code, but not the actual credentials or payload that was submitted in the request. Importantly, access logs do not record the actual **POST** body data, such as submitted credentials, so investigators can only see that the request occurred. **GET** requests may log full paths and query strings, but some log formats won't include them at all. It depends entirely on the server software being used and the logging configuration.

Investigation: TryBankMe, a small online banking platform, has suffered a breach. Attackers broke in and leaked sensitive customer data on a darknet forum. Management believes the intrusion began through the company's public website, and it's up to you to uncover exactly how it happened. Begin by opening up the `access.log` file on the desktop.

Answer the questions below

What is the attacker's User-Agent while performing the directory fuzz? FFUF v2.1.0

open access logs with vim . agent is FFUF

What is the name of the page on which the attacker performs a brute-force attack? /login.php

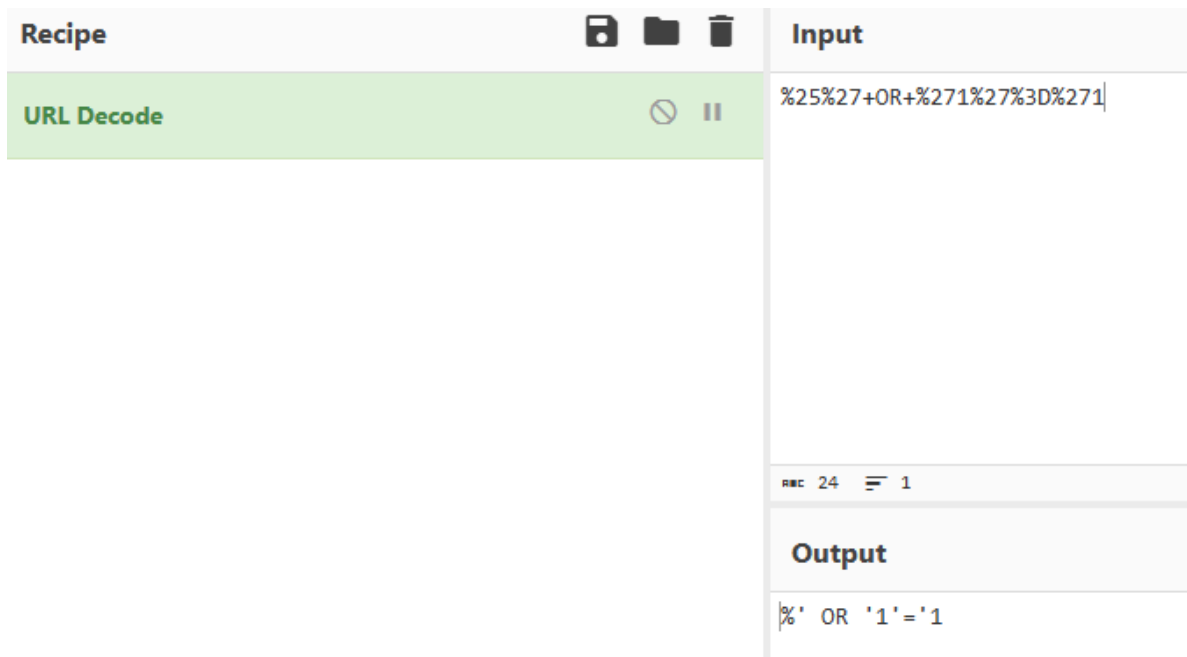
```

192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /login.php HTTP/1.1" 200 241 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /social HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /images HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /manage HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /corporate HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /company HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /files HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /home HTTP/1.1" 200 265 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /catalog HTTP/1.1" 200 335 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:38 +0000] "GET /products HTTP/1.1" 200 261 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:39 +0000] "GET /features HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:40 +0000] "GET /jobs HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:37:40 +0000] "GET /adminpanel HTTP/1.1" 404 273 "FFUF v2.1.0"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /login.php HTTP/1.0" 200 403 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /login.php HTTP/1.0" 200 403 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /login.php HTTP/1.0" 200 403 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /login.php HTTP/1.0" 200 403 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /login.php HTTP/1.0" 200 403 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /login.php HTTP/1.0" 200 403 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /login.php HTTP/1.0" 200 403 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /login.php HTTP/1.0" 200 403 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /login.php HTTP/1.0" 200 403 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /login.php HTTP/1.0" 200 403 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 200 457 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 200 457 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 200 457 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 200 457 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 200 457 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 200 457 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 200 457 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 200 457 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 302 - "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 200 457 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "POST /login.php HTTP/1.0" 200 457 "Mozilla/5.0 (Hydra)"
192.168.1.10 - - [20/Aug/2025:07:38:09 +0000] "GET /account HTTP/1.0" 301 312 "Mozilla/5.0 (Hydra)"

```

What is the complete, decoded (opens in new tab) SQLi payload the attacker uses on the /changeusername.php form? %' OR '1'='1

%25%27+OR+%271%27%3D%271



User-Agent: A header sent in HTTP requests that tells the web server what software, operating system, or tool is accessing the page. It could be human , bot, script, tool. "FFUF v2.1.0" Identifies the tool making the request: ffuf (Fuzz Faster U Fool), a popular open-source web fuzzing tool.

Task 5 - Network-Based Detection

Network traffic analysis allows analysts to examine the raw data exchanged between a client and a server. By capturing and inspecting packets, analysts can observe attack behavior on a more detailed level, including the underlying transport protocols and the application data itself. Network captures are more verbose than server logs, revealing the data behind every request and response. This can include full HTTP headers, POST bodies, cookies, and uploaded and downloaded files, for example.

Attacks in Network Traffic: Recall the sequence of events.

1. Directory fuzz to find valid directories or forms
2. Brute-force attempt with packet 13 being a successful login
3. SQL injection attempts

No.	Time	Source	Destination	Protocol	Length	User-Agent	Info
1	05:45:26	10.10.10.100	10.10.20.200	HTTP	273	gobuster/1.9	GET /testdir HTTP/1.1
2	05:45:26	10.10.10.100	10.10.20.200	HTTP	273	gobuster/1.9	GET /backup HTTP/1.1
3	05:45:26	10.10.10.100	10.10.20.200	HTTP	273	gobuster/1.9	GET /private HTTP/1.1
4	05:45:26	10.10.10.100	10.10.20.200	HTTP	8573	gobuster/1.9	GET /catalog HTTP/1.1
5	05:45:26	10.10.10.100	10.10.20.200	HTTP	10972	gobuster/1.9	GET /wordpress HTTP/1.1
6	05:45:28	10.10.10.100	10.10.20.200	HTTP	273	gobuster/1.9	GET /wp-console HTTP/1.1
7	05:45:28	10.10.10.100	10.10.20.200	HTTP	273	gobuster/1.9	GET /wp-admin HTTP/1.1
8	05:45:28	10.10.10.100	10.10.20.200	HTTP	1807	gobuster/1.9	GET /login.php HTTP/1.1
9	05:47:30	10.10.10.100	10.10.20.200	HTTP	4937	Hydra	POST /login.php HTTP/1.1
10	05:47:30	10.10.10.100	10.10.20.200	HTTP	4937	Hydra	POST /login.php HTTP/1.1
11	05:47:30	10.10.10.100	10.10.20.200	HTTP	4937	Hydra	POST /login.php HTTP/1.1
12	05:47:30	10.10.10.100	10.10.20.200	HTTP	4937	Hydra	POST /login.php HTTP/1.1
13	05:47:30	10.10.10.100	10.10.20.200	HTTP	5036	Hydra	POST /login.php HTTP/1.1
14	05:47:30	10.10.10.100	10.10.20.200	HTTP	12345	Hydra	GET /account HTTP/1.1
15	05:48:13	10.10.10.100	10.10.20.200	HTTP	1861	sqlmap/1.7.2	GET /search HTTP/1.1
16	05:49:24	10.10.10.100	10.10.20.200	HTTP	1861	sqlmap/1.7.2	GET /search HTTP/1.1

Wireshark · Follow HTTP Stream (tcp.stream eq 13) · traffic_capture.pcap

POST /login.php HTTP/1.1 ← Web Request

Host: 10.10.20.200

User-Agent: Hydra ← User-Agent

Content-Length: 45

Content-Type: application/x-www-form-urlencoded

Cookie: acookie=A%20Cookie%20check

log=admin&pwd=password123&submit=Log+In ← Username and Password Attempted

HTTP/1.1 302 Found ← Response

Date: Wed, 12 Aug 2025 05:47:30 GMT

Server: Apache/2.4.52 (Ubuntu)

Expires: Wed, 11 Jan 1984 06:00:00 GMT

Cache-Control: no-cache, must-revalidate, max-age=0, no-store, private

Set-Cookie: acookie=A%20Cookie%20check; path=/; HttpOnly

X-Frame-Options: SAMEORIGIN

Referrer-Policy: strict-origin-when-cross-origin

X-Redirect-By: Website

Location: http://10.10.20.200/account

Content-Length: 0

Connection: close

Content-Type: text/html; charset=UTF-8

Wireshark · Follow HTTP Stream (tcp.stream eq 15) · traffic_capture.pcap

GET /search?q=' OR '1'=1&Submit=Submit HTTP/1.1 ← Web Request and Payload

Host: 10.10.20.200

User-Agent: sqlmap/1.7.2 ← User-Agent

Connection: keep-alive

Referrer: http://10.10.20.200/account

Cookie: PHPSESSID=12345678901234567890

HTTP/1.1 200 OK ← Response

Date: Wed, 12 Aug 2025 05:48:13 GMT

Server: Apache/2.4.52 (Ubuntu)

Expires: Wed, 11 Jun 1984 06:00:00 GMT

Cache-Control: no-cache, must-revalidate

Content-Length: 1509

Connection: Keep-Alive

Content-Type: text/html; charset=utf-8

<!DOCTYPE html>

<html lang="en-GB">

</form>

Users Table in HTML

```
<pre>ID: ' OR '1'=1<br />First name: admin<br />Surname: admin</pre>
<pre>ID: ' OR '1'=1<br />First name: Gordon<br />Surname: Brown</pre>
<pre>ID: ' OR '1'=1<br />First name: Jane<br />Surname: Lee</pre>
<pre>ID: ' OR '1'=1<br />First name: Bob<br />Surname: Smith</pre>
```

</div>

What the Attacker Saw

```
ID: ' OR '1'=1
First name: admin
Surname: admin

ID: ' OR '1'=1
First name: Gordon
Surname: Brown

ID: ' OR '1'=1
First name: Jane
Surname: Lee

ID: ' OR '1'=1
First name: Bob
Surname: Smith
```

Answer the questions below

What password does the attacker successfully identify in the brute-force attack?

astrongpassword123

Apply **Admin** as a column, identify the password attack, and filter by `http` in Wireshark. Finally, use the filter `http && http.user_agent` and apply it as a column.

`http.request.method == POST`

Value	Info
admin,password123	POST /login.php HTTP/1.0
admin,password1	POST /login.php HTTP/1.0
admin,password	POST /login.php HTTP/1.0
admin,passwerd	POST /login.php HTTP/1.0
admin,testpass	POST /login.php HTTP/1.0
admin,summertime	POST /login.php HTTP/1.0
admin,password	POST /login.php HTTP/1.0
admin,	POST /login.php HTTP/1.0
admin,astrongpassword123	POST /login.php HTTP/1.0
admin,p@ssw0rd	POST /login.php HTTP/1.0
admin,summertime2002	POST /login.php HTTP/1.0

`http.request.method == POST` or `http.response`

```
Info
HTTP/1.1 200 OK (text/html)
HTTP/1.1 200 OK (text/html)
POST /login.php HTTP/1.0 (application/x-www-form
POST /login.php HTTP/1.0 (application/x-www-form
HTTP/1.1 200 OK (text/html)
HTTP/1.1 200 OK (text/html)
POST /login.php HTTP/1.0 (application/x-www-form
HTTP/1.1 200 OK (text/html)
POST /login.php HTTP/1.0 (application/x-www-form
HTTP/1.1 200 OK (text/html)
POST /login.php HTTP/1.0 (application/x-www-form
HTTP/1.1 200 OK (text/html)
POST /login.php HTTP/1.0 (application/x-www-form
HTTP/1.1 200 OK (text/html)
POST /login.php HTTP/1.0 (application/x-www-form
HTTP/1.1 200 OK (text/html)
POST /login.php HTTP/1.0 (application/x-www-form
HTTP/1.1 200 OK (text/html)
POST /login.php HTTP/1.0 (application/x-www-form
HTTP/1.1 302 Found
POST /login.php HTTP/1.0 (application/x-www-form
HTTP/1.1 200 OK (text/html)
HTTP/1.1 200 OK (text/html)
HTTP/1.1 301 Moved Permanently (text/html)
HTTP/1.1 200 OK (text/html)
HTTP/1.1 200 OK (text/html)
```

As shown in the image below, a **POST** request with a specific User-Agent successfully logs in. Checking the **Hypertext Transfer Protocol** section reveals both the admin username and password.

tcp.stream eq 32

No.	Time	Source	Destination	Protocol	Length	Text item	Info
308	2025-08-20 07:38:09.812486	192.168.1.10	192.168.1.9	TCP	74	✓	33994 → 80 [SYN] Seq=0 Win=64240 Len=0
314	2025-08-20 07:38:09.813927	192.168.1.9	192.168.1.10	TCP	74	✓	80 → 33994 [SYN, ACK] Seq=0 Ack=1 Win=64256
315	2025-08-20 07:38:09.813995	192.168.1.10	192.168.1.9	TCP	66	✓	33994 → 80 [ACK] Seq=1 Ack=1 Win=64256
316	2025-08-20 07:38:09.814090	192.168.1.10	192.168.1.9	HTTP	303	✓	POST /login.php HTTP/1.0 (application/x-www-form-urlencoded)
322	2025-08-20 07:38:09.815245	192.168.1.9	192.168.1.10	TCP	66	✓	80 → 33994 [ACK] Seq=1 Ack=238 Win=6502
327	2025-08-20 07:38:09.816612	192.168.1.9	192.168.1.10	HTTP	365	✓	HTTP/1.1 302 Found
328	2025-08-20 07:38:09.816613	192.168.1.9	192.168.1.10	TCP	66	✓	80 → 33994 [FIN, ACK] Seq=300 Ack=238 Win=64
329	2025-08-20 07:38:09.816666	192.168.1.10	192.168.1.9	TCP	66	✓	33994 → 80 [ACK] Seq=238 Ack=300 Win=64
377	2025-08-20 07:38:09.857873	192.168.1.10	192.168.1.9	TCP	66	✓	33994 → 80 [ACK] Seq=238 Ack=301 Win=64
400	2025-08-20 07:38:09.917264	192.168.1.10	192.168.1.9	TCP	66	✓	33994 → 80 [FIN, ACK] Seq=238 Ack=301 Win=64
402	2025-08-20 07:38:09.918266	192.168.1.9	192.168.1.10	TCP	66	✓	80 → 33994 [ACK] Seq=301 Ack=239 Win=65

Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Times...

Timestamps

SEQ/ACK analysis

TCP payload (237 bytes)

Hypertext Transfer Protocol

POST /login.php HTTP/1.0\r\n

Host: 192.168.1.9\r\n

User-Agent: Mozilla/5.0 (Hydra)\r\n

Content-Length: 42\r\n

Content-Type: application/x-www-form-urlencoded\r\n

Cookie: PHPSESSID=0137k6rt2ivjj8tdthlclfk7\r\n

\r\n

[Full request URI: http://192.168.1.9/login.php]

[HTTP request 1/1]

[Response in frame: 327]

File Data: 42 bytes

HTML Form URL Encoded: application/x-www-form-urlencoded

Form item: "username" = "admin"

Form item: "password" = "astronpassword123"

Wireshark · Follow TCP Stream (tcp.stream eq 32) · traffic.pcap

POST /login.php HTTP/1.0

Host: 192.168.1.9

User-Agent: Mozilla/5.0 (Hydra)

Content-Length: 42

Content-Type: application/x-www-form-urlencoded

Cookie: PHPSESSID=0137k6rt2ivjj8tdthlclfk7

username=admin&password=astronpassword123

HTTP/1.1 302 Found

Date: Wed, 20 Aug 2025 07:38:09 GMT

Server: Apache/2.4.52 (Ubuntu)

Expires: Thu, 19 Nov 1981 08:52:00 GMT

Cache-Control: no-store, no-cache, must-revalidate

Pragma: no-cache

Location: /account

Content-Length: 0

Connection: close

Content-Type: text/html; charset=UTF-8

What is the flag the attacker found in the database using SQLi?

THM{dumped_the_db}

350	2025-08-20	07:38:09.846394	192.168.1.9	192.168.1.10	HTTP	824	✓	HTTP/1.1 200 OK (text/html)
362	2025-08-20	07:38:09.850413	192.168.1.9	192.168.1.10	HTTP	824	✓	HTTP/1.1 200 OK (text/html)
405	2025-08-20	07:38:09.918432	192.168.1.10	192.168.1.9	HTTP	260	✓	GET /account HTTP/1.0
409	2025-08-20	07:38:09.920425	192.168.1.9	192.168.1.10	HTTP	605	✓	HTTP/1.1 301 Moved Permanently (t
422	2025-08-20	07:38:10.022690	192.168.1.10	192.168.1.9	HTTP	261	✓	GET /account/ HTTP/1.0
424	2025-08-20	07:38:10.024789	192.168.1.9	192.168.1.10	HTTP	629	✓	HTTP/1.1 200 OK (text/html)
440	2025-08-20	07:38:21.006112	192.168.1.10	192.168.1.9	HTTP	572	✓	GET /account/changeusername.php?q=
442	2025-08-20	07:38:21.009642	192.168.1.9	192.168.1.10	HTTP	717	✓	HTTP/1.1 200 OK (text/html)

```
Wireshark · Follow HTTP Stream (tcp.stream eq 44) · traffic.pcap
e3qk47fdig3hs3a30ta13s2i0
Upgrade-Insecure-Requests: 1
Priority: u=0, i

HTTP/1.1 200 OK
Date: Wed, 20 Aug 2025 07:38:20 GMT
Server: Apache/2.4.52 (Ubuntu)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Encoding: gzip
Content-Length: 289
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8

<h2>Change Username / Search Users</h2><form method="GET" action="
">
    Search for username: <input type="text" name="q">
    <input type="submit" value="Search">
</form><p><strong>DEBUG: SQL Query:</strong> SELECT id, user
name, email FROM users WHERE username LIKE '%%' OR '1'='1'</p>1 |
alice | alice@example.com<br>2 | bob | bob@example.com<br>3 | admin
n | admin@example.com<br>4 | flag | THM{dumped_the_db}<br>
```

Task 6 - Web Application Firewall

Web Application Firewalls (WAFs) are often the first line of defense for websites and web applications. WAFs act as gatekeepers for your web applications, inspecting full request packets, similar to Wireshark but with the ability to decrypt TLS traffic and filter it before it reaches the server.

Rules:

Rule Type	Description	Example Use Case
Block common attack patterns	Blocks known malicious payloads and indicators	Block malicious User-Agents: <code>sqlmap</code>
Deny known malicious sources	Uses IP reputation, threat intel, or geo-blocking to stop risky traffic	Block IPs from recent botnet campaigns
Custom-built rules	Tailored to your specific application's needs	Allow only GET/POST requests to <code>/login</code>

Rule Type	Description	Example Use Case
Rate-limiting & abuse prevention	Limits request frequency to prevent abuse	Limit login attempts to 5 per minute per IP



Answer the questions below

- | What do WAFs inspect and filter? Web Requests
 - | Create a custom firewall rule to block any User-Agent that matches "BotTHM".
- IF User-Agent CONTAINS "BotTHM" THEN block

Task 7 - Conclusion

This room covers web attack detection—from client and server-side threats to log, network, and WAF analysis—showing how to correlate data sources for a stronger defense.